Intro to Algorithms — Week 3

Divide and Conquer

Lecture notes for the demo workspace of UniHelperProMax.

Today

1. The divide-and-conquer pattern
2. Merge sort and its recurrence
3. Solving recurrences: the recursion tree
4. The master theorem
5. Binary search as a degenerate case

The pattern

Divide: split the problem into smaller instances of the same problem.
Conquer: solve each smaller instance recursively.
Combine: merge the partial answers into an answer for the whole.
Base case: stop when the instance is small enough to solve directly.

Merge sort

MergeSort(A, lo, hi):

```
if hi - lo <= 1: return
mid = (lo + hi) // 2
MergeSort(A, lo, mid)
MergeSort(A, mid, hi)
Merge(A, lo, mid, hi)
```

Merge walks both sorted halves once, so it costs Theta(n).
The running time satisfies T(n) = 2 T(n/2) + Theta(n).
Merge sort is stable and needs Theta(n) extra memory.

Why it is fast

Each level of the recursion does Theta(n) work in total,
and there are about log2(n) levels before the pieces have size 1.

Recursion trees

Draw one node per recursive call and write its local cost inside.
Sum the costs level by level, then add up the levels.

For T(n) = 2 T(n/2) + cn:
level 0: cn level 1: 2 * c(n/2) = cn level 2: 4 * c(n/4) = cn
There are log2(n) + 1 levels, so T(n) = cn log2(n) + cn = Theta(n log n).

For T(n) = 3 T(n/4) + cn^2 the level costs shrink geometrically:
cn^2, (3/16) cn^2, (3/16)^2 cn^2, ... so the root dominates: Theta(n^2).

The master theorem

For T(n) = a T(n/b) + f(n) with a >= 1 and b > 1, compare f(n) with n^(log_b a):

Case 1: f(n) = O(n^(log_b a - e)) for some e > 0 => T(n) = Theta(n^(log_b a)).
Case 2: f(n) = Theta(n^(log_b a)) => T(n) = Theta(n^(log_b a) log n).
Case 3: f(n) = Omega(n^(log_b a + e)) and a f(n/b) <= k f(n), k < 1 => T(n) = Theta(f(n)).

Examples

Merge sort: a = 2, b = 2, f(n) = n -> case 2, Theta(n log n).
Binary search: a = 1, b = 2, f(n) = 1 -> case 2, Theta(log n).
Strassen: a = 7, b = 2, f(n) = n^2 -> case 1, Theta(n^2.81).

Exercise for Wednesday

Solve T(n) = 4 T(n/2) + n^2 and T(n) = T(n/3) + T(2n/3) + n.